HERMES · PHASE 1 · LESSON 0007 · LAB EXERCISE 06-TASKSPEC · ~45–60 MIN

The TaskSpec Is a Contract

Exercise 05 called the model with a loose string and a hardcoded token limit. Who decided that job was worth running?

Why this matters for Hermes

Scenario step S1 (the envelope check) parses the job first, so Hermes rejects invalid work before it spends a token. Lesson 0005 built that parse and aimed it at a provider's reply. Today the same parse guards the operator's request. The boundary rule now covers both directions into the loop.

1 · The request nobody checked

Open exercise 05's `main.ts` and find where the job was decided. The prompt is the literal `"Hello, Claude!"`. The size of the call is `maxTokens: 100`, written inside `supervisor.ts`. No file records who asked, what the budget was, or where the answer should land.

A running Hermes takes those decisions from an operator, over a command line or a job file. Bytes from an operator are untrusted in the way a provider's reply is untrusted. Nothing in exercise 05 checks them.

Layer	Changed?	Evidence
the wire	no	Part A sends exercise 05's request, with a larger `max_tokens`
the six responsibilities ①–⑥	no	the SDK still runs all six, below the port
the port and both gateways	no	no code changed in `gateway.ts` or in either implementation
where a call's size is decided	yes	the spec file supplies it, not `supervisor.ts`
what a bad request costs	yes	Part B refuses two spec files and issues zero requests
what a report can name	yes	the task, its declared budget, and its tool list

New today.

Four ideas are new.

- A schema can describe work, not only bytes from a wire.
- A parse can decide whether work runs at all.
- `.default()` gives one schema two static types.
- A rule can span two fields, so no single field can carry it.

Reprise: `safeParse`, path-addressed issues, `z.infer`, narrowing a union on its tag, and the fake's `calls` array.

2 · A schema for the work

A *TaskSpec* is Hermes's schema for one unit of work, written before any model call. It guards a boundary in the same sense lesson 0005's schema did. The bytes reaching it come from an operator instead of a provider.

The course used to call three different things a contract. It now calls one thing a contract. The TaskSpec constrains which work Hermes may run at all. Responsibility ④ is the request and response *shape*, and `gateway.ts` is *the port*. Lessons 0001 to 0005 still carry the older wording until they are rewritten.

```
export const TaskSpecSchema = z.object({
  title:            z.string().min(1),
  owner:            z.string().min(1),
  instruction:      z.string().min(1),        // becomes ModelCall.prompt
  maxTokens:        z.number().int().positive().max(4096).default(1024),
  costCeilingTokens: z.number().int().positive(),
  allowedTools:     z.array(z.string()).default([]),
  outputPath:       z.string().min(1),
}).refine((spec) ⇒ spec.maxTokens ≤ spec.costCeilingTokens, {
  error: "maxTokens may not exceed the job's costCeilingTokens",
  path: ["maxTokens"],
});

export type TaskSpec      = z.output<typeof TaskSpecSchema>;
export type TaskSpecInput = z.input<typeof TaskSpecSchema>;
```

Syntax	Meaning	At runtime?
`.default(1024)`	the field may be absent on the way in	a value the parse writes when the key is missing
`z.input<…>`	the type a spec file may have	nothing. TypeScript erases it
`z.output<…>`	the type an admitted spec has	nothing. The parsed value carries the guarantee
`.refine(fn, …)`	a rule over the whole object	a function the parse calls after the fields pass

A signature tells TypeScript what may be called. The implementation does the runtime work. One declaration here produces two signatures and one implementation. Lesson 0005 needed only `z.infer`, because that schema had no defaults, and its two types were identical.

Part A measures what the parse added. The file carries five keys and the admitted spec has seven.

```
// audit-atlas.json, 5 keys           // the admitted spec, 7 keys
{                                     {
  title: "Audit the atlas graph",       title: "Audit the atlas graph",
  owner: "themis",                      owner: "themis",
  instruction: "Summarize the …",       instruction: "Summarize the …",
  costCeilingTokens: 2000,              maxTokens: 1024,          // filled
  outputPath: "vault/audits/…",         costCeilingTokens: 2000,
}                                       allowedTools: [],         // filled
                                        outputPath: "vault/audits/…",
                                      }
```

Lesson 0005 counted what a parse stripped, and this parse adds. Both counts say the same thing. The contract is the parsed value, never the file. The `allowedTools` default is an empty array. A permission list that grants nothing is the one you can ship before lesson 0010 enforces it.

A default that no file mentions still reaches the network. Part A's request, captured off the wire:

```
POST /v1/messages
anthropic-version: 2023-06-01 · x-api-key: … · content-type: application/json
user-agent: Anthropic/JS 0.113.0

{"model":"claude-opus-4-8","max_tokens":1024,
 "messages":[{"role":"user",
              "content":"Summarize the atlas graph health check in three bullet points."}]}
```

The file named no `maxTokens`, and `max_tokens` on the wire reads 1024. The adapter read it from the spec, and exercise 05 read 100 from a literal.

3 · The refusal costs nothing

An *admissibility check* is the parse that decides whether Hermes may dispatch a job. `admitTaskSpec` is that check. It returns the union shape you narrowed in lesson 0005 for `safeParse`, and again in lesson 0006 for `GatewayResult`.

```
export type Admission =
  | { admitted: true;  spec: TaskSpec }
  | { admitted: false; rejections: string[] };
```

`runTask` asks for a `TaskSpec`, and a successful parse is the only way to obtain one. A caller that skips the check fails to compile. S1's rule therefore holds at build time and at runtime, by two different mechanisms.

```
no-owner.json → REFUSED, 3 reason(s):
  · title: [too_small] Too small: expected string to have ≥1 characters
  · owner: [invalid_type] Invalid input: expected string, received undefined
  · costCeilingTokens: [invalid_type] Invalid input: expected number, received string

not-a-spec.json → REFUSED, 1 reason(s):
  · (root): [invalid_type] Invalid input: expected object, received array

model calls made: 0 — tokens spent: 0
```

One parse reported every disagreement in the file, which is lesson 0005's behaviour aimed at a new boundary. The mock's counter proves the cost. Against a fresh mock, Part A takes `req_mock_0001`, a full Part B follows, and the next Part A takes `req_mock_0002`. No request reached the server while the check refused those files.

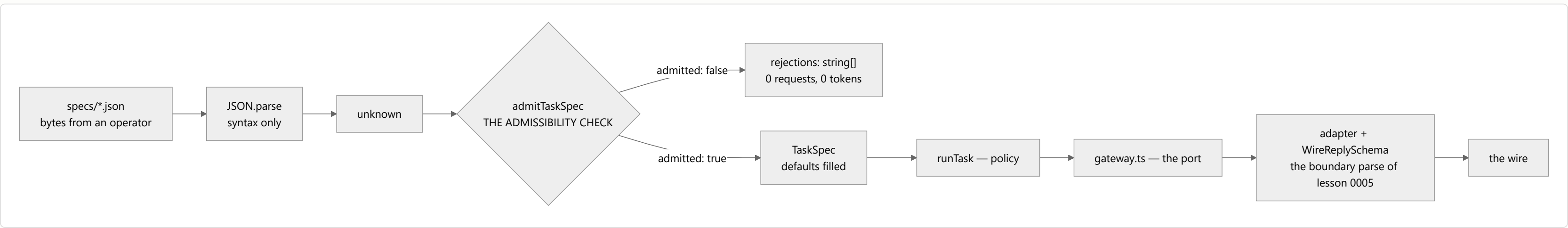

Diagram 1. Two boundaries, two parses, one loop. The spec is checked on the way in, and the reply is checked on the way back.

4 · The rule no single field can carry

A *refinement* is a schema rule that runs after every field's type passes. `over-ceiling.json` sets `maxTokens` to 4000 against a `costCeilingTokens` of 1000. Both are positive integers, and both sit inside their declared bounds. The file is still wrong, and `z.object` cannot say so, because it checks one field at a time.

```
over-ceiling.json → REFUSED:
  · maxTokens: [custom] maxTokens may not exceed the job's costCeilingTokens
```

The issue code reads `custom`, because you wrote the rule rather than declaring a type. The path reads `maxTokens`, because you chose it. Drop the `path` option and the issue lands at the root of the object, which names no field an operator can edit.

Order matters here, and it is measurable. A file that omits `owner` and also breaks the ceiling rule reports the `owner` issue alone. The refinement never runs, so one bad file can be refused twice, in two rounds.

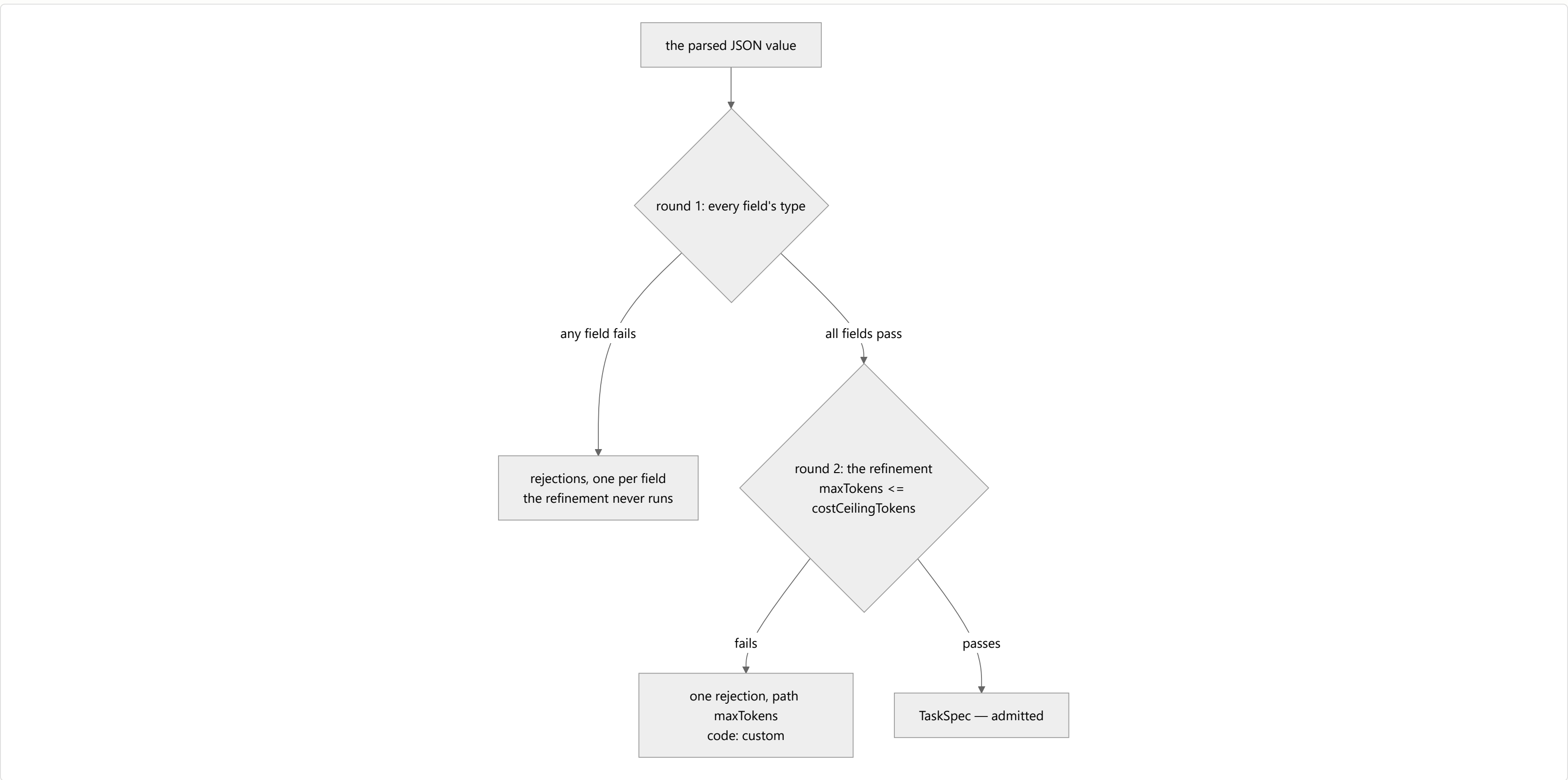

Diagram 2. The two rounds, measured in Part C. A spec that is wrong in both rounds is rejected twice, and reports round 1 first.

Capability	Status	Where it lives, or when it arrives
A parsed spec describes the work	implemented	`task-spec.ts`, Parts A to C
Invalid work costs zero requests	implemented	Part B, the fake's `calls` and the mock's counter
A token budget per job	prepared	`costCeilingTokens` is carried and reported. Lesson 0009 enforces it
A tool permission list	prepared	`allowedTools` is carried and defaults to empty. Lesson 0010 enforces it
The artifact lands at `outputPath`	planned	No `ROADMAP.md` row writes a file yet. That is S8 (landing the outputs)

5 · The exercise

Commands and the measured table live in `06-taskspec/README.md`. The arc:

1. Read `task-spec.ts` against `gateway.ts`. Say what each file constrains, and which one the model ever sees.
2. Run Part A. Compare the file's key list with the admitted spec's key list.
3. Uncomment the marked line in Part A. Read the error, then say which of the two types produced it.
4. Implement `admitTaskSpec`, formatting issues the way the adapter formats its own. Run Part B and read all four rejections.
5. Run Part B, then Part A, and check the request id the mock hands out. That number is the measurement.

6 · Exercise: which boundary refuses it?

Place each failure with the check that catches it

Answer from memory before you click. Feedback is immediate.

A spec file whose `costCeilingTokens` reads `"2000"`.

the admissibility check, before dispatch | the adapter's boundary parse, after the reply | the compiler, before the run | nothing refuses it today

A reply whose `output_tokens` arrives as `"42"`.

the admissibility check, before dispatch | the adapter's boundary parse, after the reply | the compiler, before the run | nothing refuses it today

Code that hands `runTask` the object it read straight off disk.

the admissibility check, before dispatch | the adapter's boundary parse, after the reply | the compiler, before the run | nothing refuses it today

A spec whose `maxTokens` is 4000 against a ceiling of 1000.

the admissibility check, before dispatch | the adapter's boundary parse, after the reply | the compiler, before the run | nothing refuses it today

A job that runs on and spends more tokens than its declared ceiling.

the admissibility check, before dispatch | the adapter's boundary parse, after the reply | the compiler, before the run | nothing refuses it today

A spec listing `"graph.delete"`, a tool the loop has never implemented.

the admissibility check, before dispatch | the adapter's boundary parse, after the reply | the compiler, before the run | nothing refuses it today

Score: 0 / 0

7 · Quiz

Four questions

Q1. What does `.default(1024)` change about the schema's two static types?

- Nothing at all, because a default is a runtime value with no type
- It makes the field optional on both sides, so the loop must check
- The input type may omit the field, and the output type may not
- It widens the field to a union of number and undefined everywhere

Q2. Part B refused two spec files. What is the evidence that nothing was spent?

- The reports each carried an outcome of gave_up and a token count of zero
- The fake recorded no calls, and the mock's request counter did not advance
- The adapter returned a malformed_reply failure before the request was sent
- The supervisor caught each error and rolled the job's ledger back to zero

Q3. Every field in `over-ceiling.json` has a valid type. Why is the file refused?

- A refinement runs after the fields, and this rule reads two of them
- The declared maximum of 4096 rejects any value above the job's ceiling
- The schema strips unknown keys, and the extra tool entry is one of them
- A default cannot be written once the file already supplies that same key

Q4. `runTask` takes a `TaskSpec`. What stops a caller from passing an unparsed object?

- A runtime guard at the top of runTask re-parses whatever it was given
- Nothing does, so the convention is documented in the exercise's README
- The port refuses the call, because gateway.ts imports the spec's schema
- Only a successful parse yields that type, so the call fails to compile

Score: 0 / 0

8 · Say it in your own words

Answer these in the chat after the lab, with no notes.

1. Name the two boundaries this exercise parses. Say which direction each one guards, and what arrives at it.
2. A reviewer says the ceiling rule belongs in `runTask`, not in the schema. Answer in two sentences.
3. Explain why `admitTaskSpec` takes `unknown` rather than `TaskSpecInput`.

If you keep only five sentences from this lesson, keep these. A TaskSpec is a schema for the work, and Hermes parses it before the loop calls any model. The admissibility check returns proof or reasons as data, and it refuses without throwing. A refused spec costs zero requests, and Part B measured that against the mock's counter. A default value splits one schema into an input type a file may hold and an output type the loop requires.

Primary source

Read Alexis King's Parse, don't validate, the essay this lesson's gate comes from. Its central move is the one `runTask` makes: put the guarantee in the type, so the unchecked value cannot travel. Then read Zod's API reference on defaults and refinements.

Version pin. Every excerpt and number here was verified against `zod` 4.4.3 and `@anthropic-ai/sdk` 0.113.0, by live runs against the exercise 01 mock. The three new terms have notes under `wiki/terms/`.

Next. Lesson 0008, *Tool Use, the Loop's Heartbeat*, with lab `07-tool-loop`. The spec's `allowedTools` field gets its first reader there, and one call becomes a loop.